

1BM24CS315-LAB-2

QUESTION:

Implement 8-puzzle problem using (i) Depth First Search (ii) Iterative Deepening search

Solution: (i)

```
GOAL_STATE = (1, 2, 3, 4, 5, 6, 7, 8, 0)

class PuzzleNode:
    def __init__(self, state, parent=None, action=None, depth=0):
        self.state = state
        self.parent = parent
        self.action = action
        self.depth = depth

    def get_blank_index(self):
        return self.state.index(0)

    def generate_successors(self):
        successors = []
        idx = self.get_blank_index()
        row, col = idx // 3, idx % 3

        moves = [(-1, 0, 'Up'), (1, 0, 'Down'), (0, -1, 'Left'), (0, 1, 'Right')]

        for dr, dc, action in moves:
            new_row, new_col = row + dr, col + dc
            if 0 <= new_row < 3 and 0 <= new_col < 3:
                new_idx = new_row * 3 + new_col
                state_list = list(self.state)
                # Swap elements to create the new board state
                state_list[idx], state_list[new_idx] = state_list[new_idx], state_list[idx]

                successor_node = PuzzleNode(
                    state=tuple(state_list),
                    parent=self,
                    action=action,
                    depth=self.depth + 1
                )
                successors.append(successor_node)
        return successors

    def get_path(self):
        node, path = self, []
        while node:
```

```

        path.append((node.action, node.state))
        node = node.parent
    return path[::-1]

def print_board(state):
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]} ")
    print("-" * 11)

def depth_first_search(start_state, max_depth_limit=30):
    start_node = PuzzleNode(start_state)
    stack = [start_node]
    visited = set()

    while stack:
        node = stack.pop()

        if node.state == GOAL_STATE:
            return node.get_path()

        if node.depth < max_depth_limit:
            visited.add(node.state)
            # Reverse successors to explore them in standard order
            for successor in reversed(node.generate_successors()):
                if successor.state not in visited:
                    stack.append(successor)

    return None

if __name__ == "__main__":
    # Initial state (3 moves away from goal)
    initial_puzzle = (1, 2, 3, 4, 0, 5, 7, 8, 6)

    print("Initial Board Setup:")
    print_board(initial_puzzle)

    print("\n--- Running Depth First Search (DFS) ---")
    dfs_solution = depth_first_search(initial_puzzle, max_depth_limit=25)

    if dfs_solution:
        print(f"DFS found a solution path in {len(dfs_solution) - 1} moves.")
        for action, state in dfs_solution:
            if action:
                print(f"Action taken: Move blank tile '{action}'")
            print_board(state)
    else:
        print("No solution found within the specified depth limit.")

```

```
> python -u d:\AI\4.DFS_8_puzzle.py
```

```
Initial Board Setup:
```

```
1 2 3
4 0 5
7 8 6
```

```
-----
```

```
--- Running Depth First Search (DFS) ---
```

```
DFS found a solution path in 24 moves.
```

```
1 2 3
4 0 5
7 8 6
```

```
-----
```

```
Action taken: Move blank tile 'Up'
```

```
1 0 3
4 2 5
7 8 6
```

```
-----
```

```
Action taken: Move blank tile 'Right'
```

```
1 3 0
4 2 5
7 8 6
```

```
-----
```

```
Action taken: Move blank tile 'Down'
```

```
1 3 5
4 2 0
7 8 6
```

```
-----
```

```
Action taken: Move blank tile 'Down'
```

```
1 3 5
4 2 6
7 8 0
```

```
-----
```

```
Action taken: Move blank tile 'Left'
```

```
1 3 5
4 2 6
7 0 8
```

```
-----
```

```
Action taken: Move blank tile 'Up'
```

```
1 3 5
4 0 6
7 2 8
```

```
-----
```

```
Action taken: Move blank tile 'Up'
```

```
1 0 5
4 3 6
7 2 8
```

```
-----
```

```
Action taken: Move blank tile 'Left'
```

```
0 1 5
4 3 6
7 2 8
```

```
-----
```

```
Action taken: Move blank tile 'Left'
0 1 5
4 3 6
7 2 8
-----
Action taken: Move blank tile 'Down'
4 1 5
0 3 6
7 2 8
-----
Action taken: Move blank tile 'Right'
4 1 5
3 0 6
7 2 8
-----
Action taken: Move blank tile 'Down'
4 1 5
3 2 6
7 0 8
-----
Action taken: Move blank tile 'Right'
4 1 5
3 2 6
7 8 0
-----
Action taken: Move blank tile 'Up'
4 1 5
3 2 0
7 8 6
-----
Action taken: Move blank tile 'Left'
4 1 5
3 0 2
7 8 6
-----
Action taken: Move blank tile 'Left'
4 1 5
0 3 2
7 8 6
-----
Action taken: Move blank tile 'Up'
0 1 5
4 3 2
7 8 6
-----
Action taken: Move blank tile 'Right'
1 0 5
4 3 2
7 8 6
-----
```

```

-----
Action taken: Move blank tile 'Right'
1 0 5
4 3 2
7 8 6
-----
Action taken: Move blank tile 'Right'
1 5 0
4 3 2
7 8 6
-----
Action taken: Move blank tile 'Down'
1 5 2
4 3 0
7 8 6
-----
Action taken: Move blank tile 'Left'
1 5 2
4 0 3
7 8 6
-----
Action taken: Move blank tile 'Up'
1 0 2
4 5 3
7 8 6
-----
Action taken: Move blank tile 'Right'
1 2 0
4 5 3
7 8 6
-----
Action taken: Move blank tile 'Down'
1 2 3
4 5 0
7 8 6
-----
Action taken: Move blank tile 'Down'
1 2 3
4 5 6
7 8 0
-----

```

Solution: (ii)

```

INITIAL = (1, 2, 3,
           0, 4, 6,
           7, 5, 8)

GOAL = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

```

```

def get_neighbors(state):
    neighbors = []
    b = state.index(0)
    row, col = divmod(b, 3)

    moves = [("Up", -1, 0), ("Down", 1, 0), ("Left", 0, -1), ("Right", 0, 1)]
    for name, dr, dc in moves:
        r, c = row + dr, col + dc
        if 0 <= r < 3 and 0 <= c < 3:
            n = r * 3 + c
            new = list(state)
            new[b], new[n] = new[n], new[b]
            neighbors.append((name, tuple(new)))
    return neighbors

def is_solvable(state):
    tiles = [x for x in state if x != 0]
    inv = sum(1 for i in range(len(tiles))
              for j in range(i + 1, len(tiles)) if tiles[i] > tiles[j])
    return inv % 2 == 0

def print_state(state):
    for i in range(0, 9, 3):
        print(" ".join(str(x) if x != 0 else "_" for x in state[i:i + 3]))
    print()

def depth_limited_search(state, goal, limit, path, on_path, counter):
    counter[0] += 1
    if state == goal:
        return path
    if limit == 0:
        return "cutoff"

    cutoff_occurred = False
    for move, nxt in get_neighbors(state):
        if nxt in on_path:
            continue
        on_path.add(nxt)
        result = depth_limited_search(nxt, goal, limit - 1, path + [move],
on_path, counter)
        on_path.remove(nxt)

```

```

        if result == "cutoff":
            cutoff_occurred = True
        elif result is not None:
            return result
    return "cutoff" if cutoff_occurred else None

def ids(start, goal, max_depth=50):
    counter = [0]
    for depth in range(max_depth + 1):
        result = depth_limited_search(start, goal, depth, [], {start},
counter)
        print(f" IDS depth limit {depth:2d} -> nodes so far: {counter[0]}")
        if result != "cutoff":
            return result, counter[0]
    return None, counter[0]

def show_solution(start, path):
    state = start
    print("Initial state:")
    print_state(state)
    moves = {"Up": -3, "Down": 3, "Left": -1, "Right": 1}
    for i, m in enumerate(path, 1):
        b = state.index(0)
        n = b + moves[m]
        s = list(state)
        s[b], s[n] = s[n], s[b]
        state = tuple(s)
        print(f"Step {i}: move blank {m}")
        print_state(state)

if __name__ == "__main__":
    if not is_solvable(INITIAL):
        print("This puzzle is not solvable.")
        raise SystemExit

    print("=== IDS ===")
    path, n = ids(INITIAL, GOAL)
    print(f"Solution length: {len(path)} moves, total nodes visited: {n}")
    print("Moves:", path, "\n")
    show_solution(INITIAL, path)

```

Output:

```
=== IDS ===
  IDS depth limit 0 -> nodes so far: 1
  IDS depth limit 1 -> nodes so far: 5
  IDS depth limit 2 -> nodes so far: 14
  IDS depth limit 3 -> nodes so far: 30
Solution length: 3 moves, total nodes visited: 30
Moves: ['Right', 'Down', 'Right']

Initial state:
1 2 3
_ 4 6
7 5 8

Step 1: move blank Right
1 2 3
4 _ 6
7 5 8

Step 2: move blank Down
1 2 3
4 5 6
7 _ 8

Step 3: move blank Right
1 2 3
4 5 6
7 8 _
```